

Kubernetes Container Security Pipeline

Project 1 of 2 — Hands-On DevSecOps Portfolio

Oluwaseyi Michael Falode | Cybersecurity & Cloud Security Engineer | Toronto, ON | May 16, 2026

This document is a complete, step-by-step record of a Kubernetes container security pipeline built from scratch on May 16, 2026. Every command was run live in a real terminal. Every screenshot was captured during the actual session. Nothing was simulated.

The document is written for two audiences. If you are technical — a security engineer, DevOps engineer, or hiring manager from a security team — you will find real configuration files, real commands, and real tool output. If you are non-technical — a recruiter or someone evaluating portfolio depth — every concept is explained in plain language before the technical work begins.

Project at a Glance

Tools Used	Trivy · OPA Gatekeeper · Falco · Helm · Minikube · kubectl
Platform	macOS — Docker Desktop v29.1.3 — Minikube v1.38.1 — Kubernetes v1.35.1
Languages	YAML (configuration) · Rego (OPA policy language)
Security Layers	Layer 1: Image scanning before deployment (Trivy)
	Layer 2: Admission control at deployment (OPA Gatekeeper)
	Layer 3: Runtime threat detection during execution (Falco)
Frameworks	CIS Kubernetes Benchmark · NIST SP 800-190 Container Security
MITRE ATT&CK	T1610 Deploy Container · T1543 Create Process · T1057 Process Discovery
Date Completed	Saturday, May 16, 2026

The Problem This Project Solves

Containers are the backbone of modern application infrastructure. Banks, hospitals, retail platforms, and government systems all run their applications in containers managed by Kubernetes. The problem is that security is often an afterthought — teams deploy containers quickly and check for security issues later, if at all.

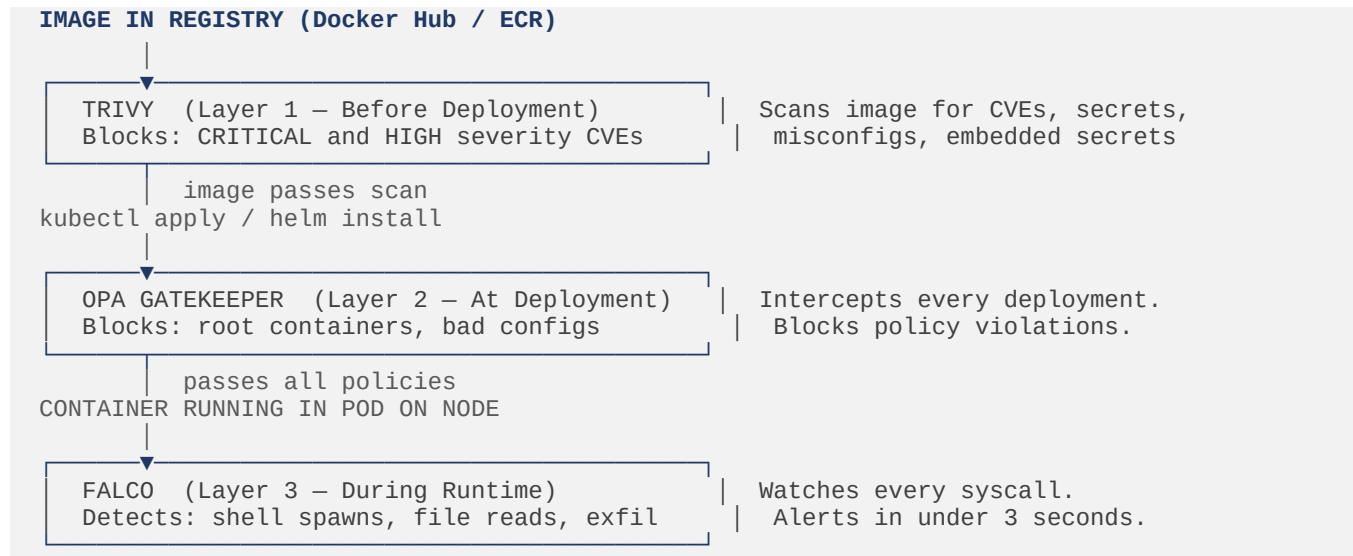
This creates three dangerous gaps, each at a different stage of the container lifecycle:

- Gap 1 — Before deployment:** The container image itself may contain known vulnerabilities inside its bundled software packages. A developer downloads a standard image from Docker Hub and deploys it without ever checking what is inside. Some of those packages may have publicly known exploits.
- Gap 2 — At deployment:** Even if the image is clean, the way the container is configured can be dangerous. A developer might configure a container to run with root (administrator) access, or give it access to the host machine's network. These misconfigurations turn a minor vulnerability into a full system breach.
- Gap 3 — During runtime:** Even if the image is clean and the configuration is correct, something can still go wrong after the container starts running. An attacker might exploit a vulnerability in the application itself and start running commands inside the container. Without runtime monitoring, this goes undetected.

This pipeline addresses all three gaps simultaneously using three purpose-built open-source tools — one for each layer of risk.

Pipeline Architecture

The three tools work as sequential checkpoints. A container must pass each checkpoint before reaching the next stage:



STEP 1 — Environment Check & Tool Installation

What we are doing in this step

Before building anything, the working environment must be verified. This is standard practice in any engineering project — you confirm what you already have and identify what is missing before writing a single line of configuration.

Tools required for this project

- **Docker Desktop** — creates and runs containers on the local machine. Acts as the engine that Minikube uses to build the cluster.
- **Minikube** — creates a miniature Kubernetes cluster on your local machine. Normally Kubernetes runs on cloud servers. Minikube simulates that environment locally inside Docker, making it ideal for development and testing.
- **kubectl** — the command-line tool used to communicate with a Kubernetes cluster. Think of it as the steering wheel for the cluster.
- **Trivy** — the image vulnerability scanner. Checks every package inside a container image against a public database of known security vulnerabilities.
- **Helm** — a package manager for Kubernetes. Similar to Homebrew for Mac or apt for Linux, but for Kubernetes applications.

Step 1A — Initial tool audit

The following commands were run to check which tools were already installed and which versions were present:

```
docker --version          # check Docker version
kubectl version --client  # check kubectl version
which trivy               # check if Trivy is installed
which minikube            # check if Minikube is installed
```

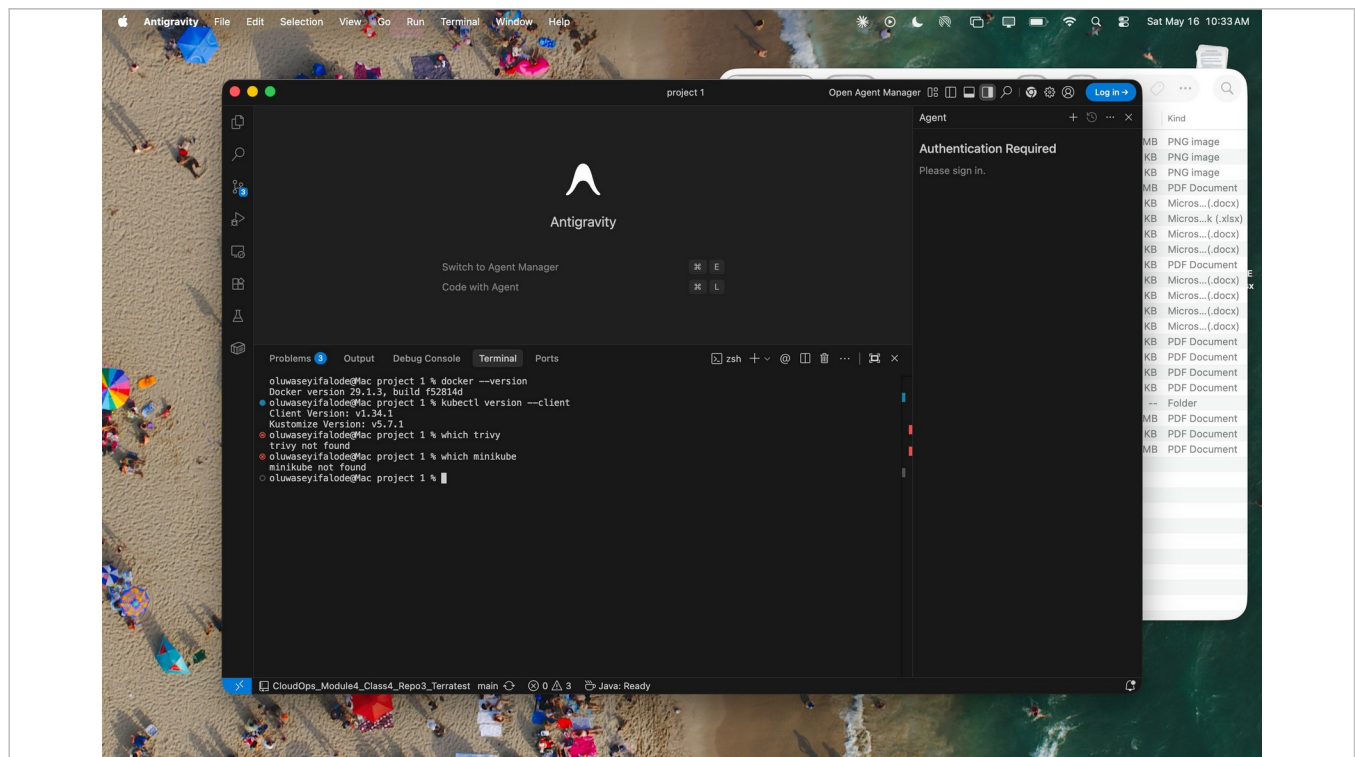


Figure 1: Tool audit results: Docker v20.10.11 installed ✓ kubectl v1.34.1 installed ✓ trivy: not found ✗ minikube: not found ✗

Step 1B — Installing missing tools

Minikube and Trivy were both installed using Homebrew — the standard package manager for macOS. Homebrew handles downloading the correct version, verifying the installation, and configuring the tool automatically.

```
brew install minikube # installs Minikube
brew install trivy     # installs Trivy
```

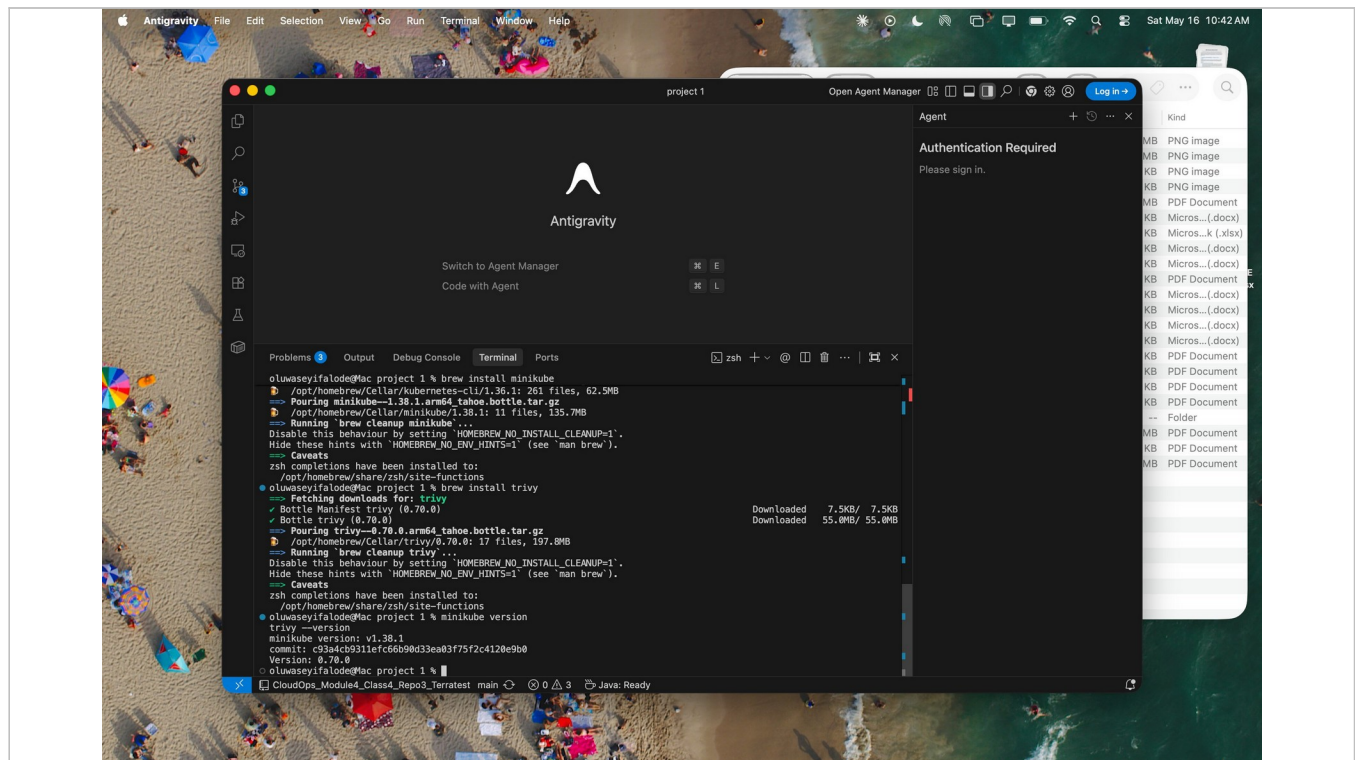


Figure 2: Both tools installed successfully. Minikube v1.38.1 ✓ Trivy v0.70.0 ✓ No errors encountered.

STEP 2 — Kubernetes Cluster Deployment

What we are doing in this step

A Kubernetes cluster is the environment where all containers will run. Normally a cluster lives on cloud infrastructure — AWS, Azure, or GCP. For this project, Minikube creates a single-node cluster running entirely inside Docker Desktop on the local machine. The behaviour is identical to a production cluster.

Key terms explained

- **Cluster:** The entire Kubernetes environment — a group of nodes working together under one control plane.
- **Node:** A single machine (physical or virtual) inside the cluster. In this project, the node is a Docker container running on the Mac.
- **Pod:** The smallest deployable unit in Kubernetes — a wrapper around one or more containers. Every container runs inside a pod.
- **Namespace:** A logical partition inside a cluster. Like floors in a building. Different teams or applications can be isolated in different namespaces.
- **kubectl:** The command-line tool used to send instructions to the cluster. Every interaction with the cluster goes through kubectl.

Step 2A — First attempt (failed — Docker Desktop not running)

The first attempt to start Minikube failed immediately. Minikube uses Docker Desktop as its engine. Docker Desktop was not running, so Minikube could not connect to it. The error message was clear: it could not find the Docker socket file, which is the communication pipe between Minikube and Docker.

This is a common real-world troubleshooting scenario. The fix was simply to open Docker Desktop, wait for it to fully start, then retry the command.

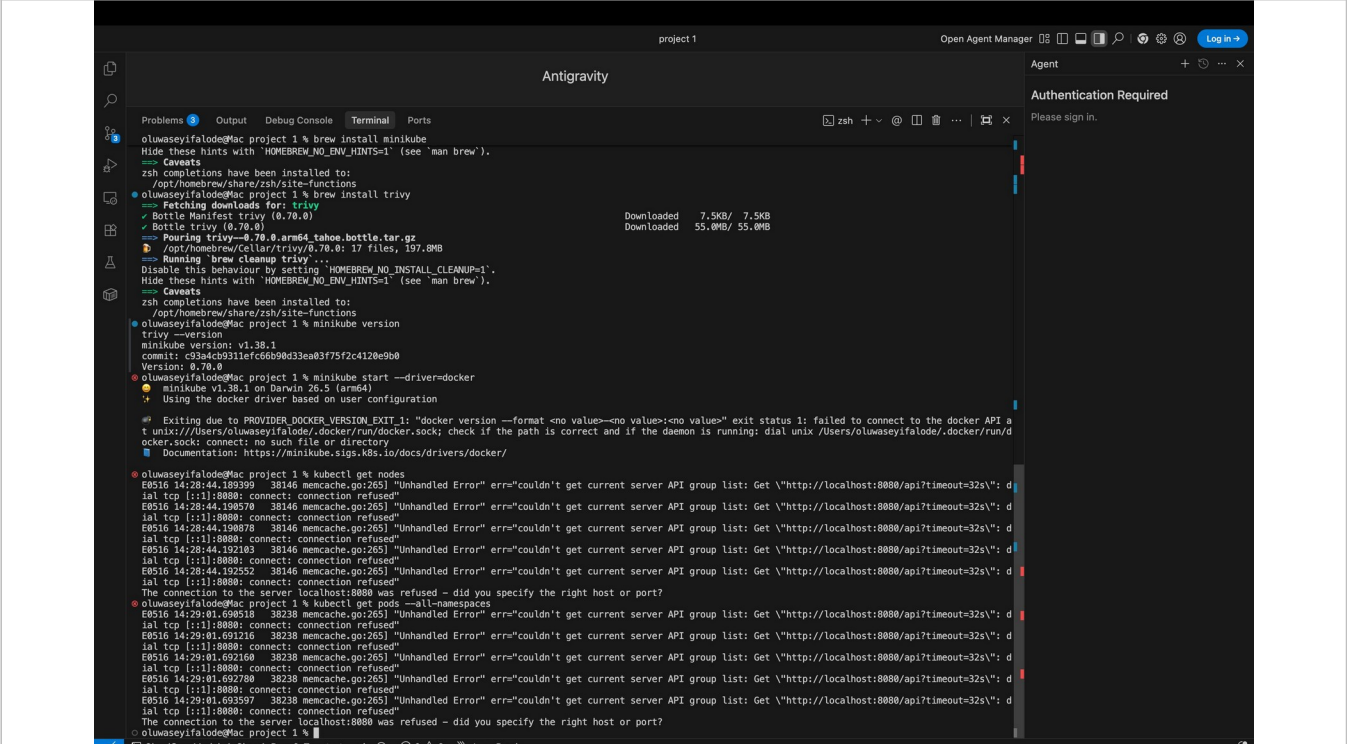


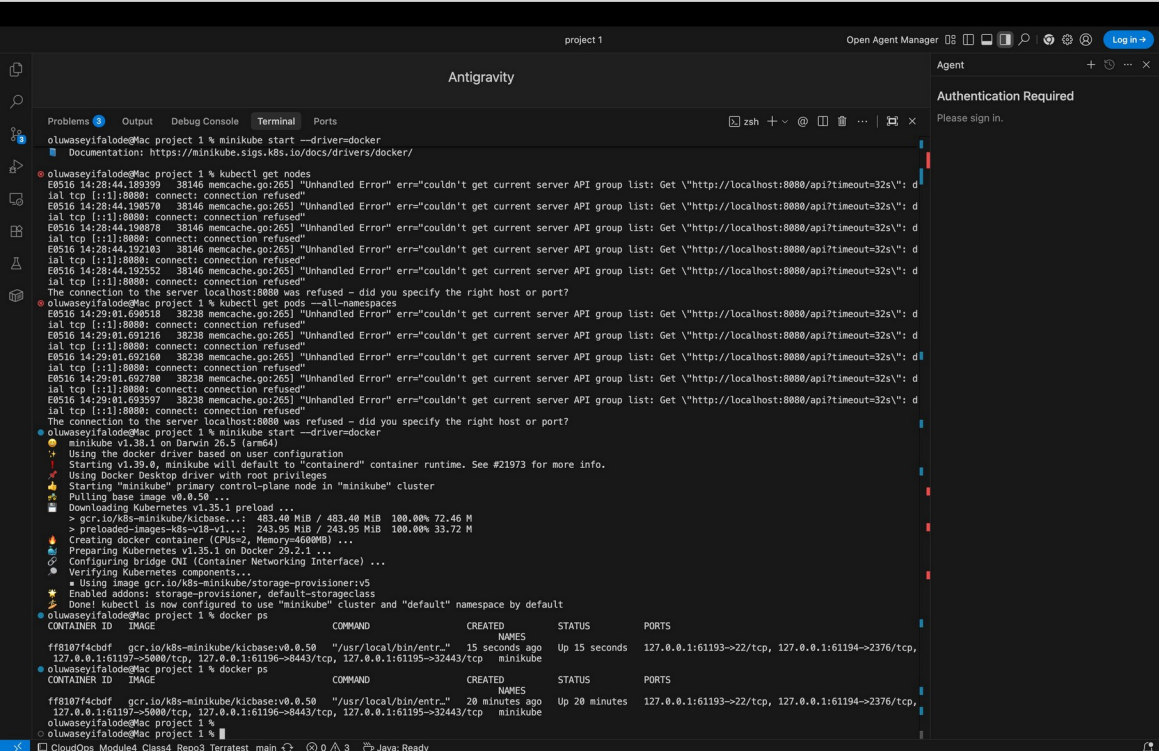
Figure 3: First minikube start attempt: FAILED. Error: failed to connect to docker API at docker.sock — Docker Desktop was not running.

Step 2B — Successful cluster start

After launching Docker Desktop, Minikube was started successfully with the following command. The `--driver=docker` flag tells Minikube to use Docker Desktop as the engine for the cluster node.

```
minikube start --driver=docker
```

What happened during startup: Minikube downloaded Kubernetes v1.35.1, created a Docker container to act as the cluster node (allocated 2 CPUs and 4600MB of memory), installed all Kubernetes internal components, and automatically configured `kubectl` to point to the new cluster.



```
project 1
Antigravity
Problems Output Debug Console Terminal Ports
oluwaseyifalode@Mac project 1 % minikube start --driver=docker
Documentation: https://minikube.sigs.k8s.io/docs/drivers/docker/

oluwaseyifalode@Mac project 1 % kubectl get nodes
E0516 14:29:44.180199 38146 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
E0516 14:28:44.190570 38146 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
E0516 14:28:44.192183 38146 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
E0516 14:28:44.190878 38146 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
E0516 14:28:44.192552 38146 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
The connection to the server localhost:8080 was refused - did you specify the right host or port?
oluwaseyifalode@Mac project 1 % kubectl get pods --all-namespaces
E0516 14:29:01.690115 38238 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
E0516 14:29:01.691216 38238 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
E0516 14:29:01.692160 38238 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
E0516 14:29:01.692780 38238 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
E0516 14:29:01.693597 38238 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": d
ial tcp [::1]:8080: connect: connection refused"
The connection to the server localhost:8080 was refused - did you specify the right host or port?
oluwaseyifalode@Mac project 1 % minikube start --driver=docker
minikube v1.38.1 on darwin (amd64)
Using the docker driver based on user configuration
Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
Using Docker Desktop driver with root privileges
Starting "minikube" primary control-plane node in "minikube" cluster
Pulling base image v0.0.50 ...
Downloading Kubernetes v1.35.1 preload ...
> gcr.io/k8s-minikube/kicbase-v0.0.50: 483.40 MiB / 483.40 MiB 100.00% 72.46 M
> preload-images-k8s-v18-v1... 243.95 MiB / 243.95 MiB 100.00% 33.72 M
Creating docker container (CPUs=2, Memory=4600MB) ...
Preparing Kubernetes v1.35.1 on Docker 23.2.1 ...
Configuring bridge CNI (Container Networking Interface) ...
Verifying Kubernetes components...
Using image gcr.io/k8s-minikube/storage-provisioner:v5
Enabled addons: storage-provisioner, default-storageclass
Done! Kubectl is now configured to use "minikube" cluster and "default" namespace by default
oluwaseyifalode@Mac project 1 % docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
ff8187f4cddf gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr..." 15 seconds ago Up 15 seconds 127.0.0.1:61193->22/tcp, 127.0.0.1:61194->2376/tcp,
oluwaseyifalode@Mac project 1 % docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
ff8187f4cddf gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr..." 20 minutes ago Up 20 minutes 127.0.0.1:61193->22/tcp, 127.0.0.1:61194->2376/tcp,
oluwaseyifalode@Mac project 1 %
oluwaseyifalode@Mac project 1 %
```

Figure 4: Minikube started successfully: Kubernetes v1.35.1 installed. Docker container running as cluster node (Up 20 minutes). `kubectl` configured automatically.

Step 2C — Verifying cluster health

Two commands were run to confirm the cluster was fully operational before proceeding.

```
kubectl get nodes # list all nodes in the cluster
kubectl get pods --all-namespaces # list all running system pods
```

The first command confirmed one node named 'minikube' with status Ready. The second showed 7 system pods all in Running state — these are Kubernetes' own internal processes (DNS, scheduler, API server, etc.) that it starts automatically to keep the cluster alive.

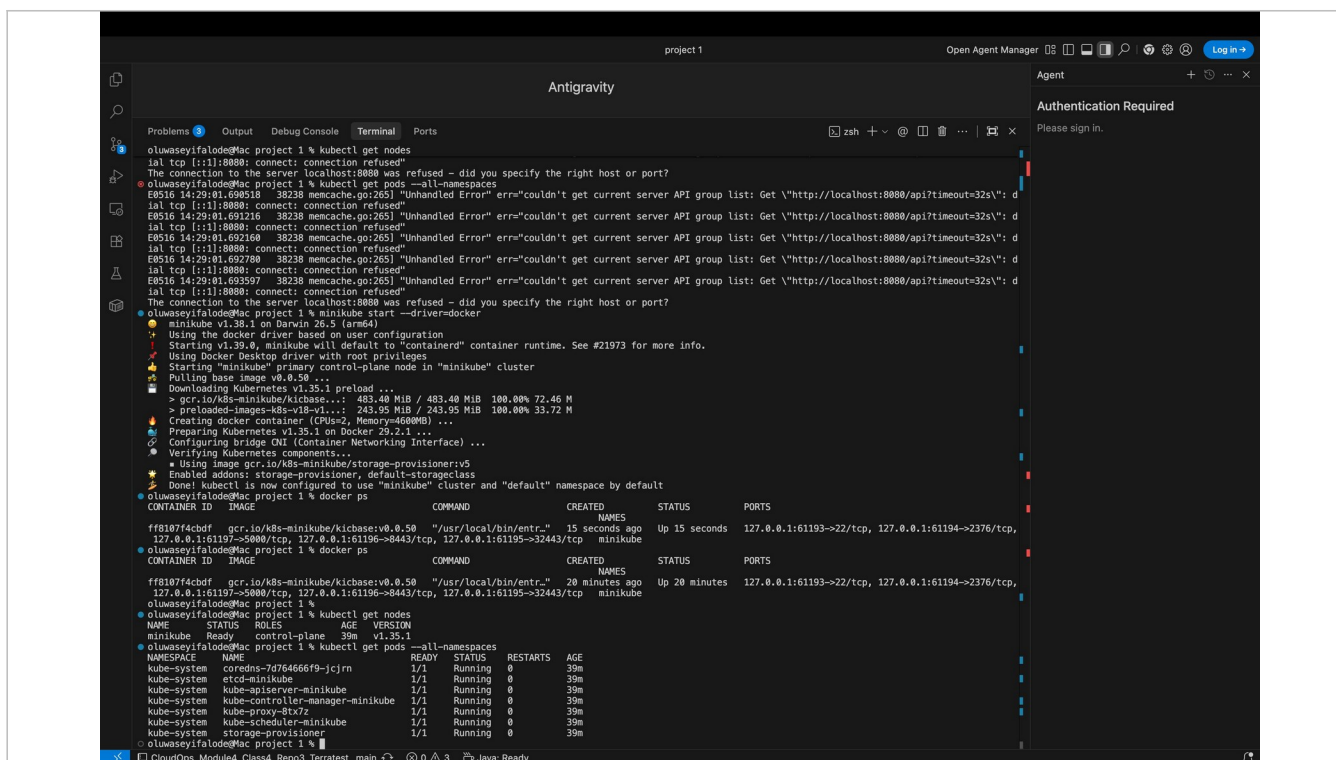


Figure 5: Cluster health verified: minikube node STATUS=Ready, VERSION=v1.35.1. All 7 system pods Running in kube-system namespace with 0 restarts.

STEP 3 — OPA Gatekeeper — Admission Control

What we are doing in this step

OPA Gatekeeper is being installed into the cluster as a security enforcement layer. Once installed, it intercepts every single deployment request before Kubernetes acts on it. If a deployment violates any security policy, Gatekeeper blocks it and returns a clear error message. The deployment never happens.

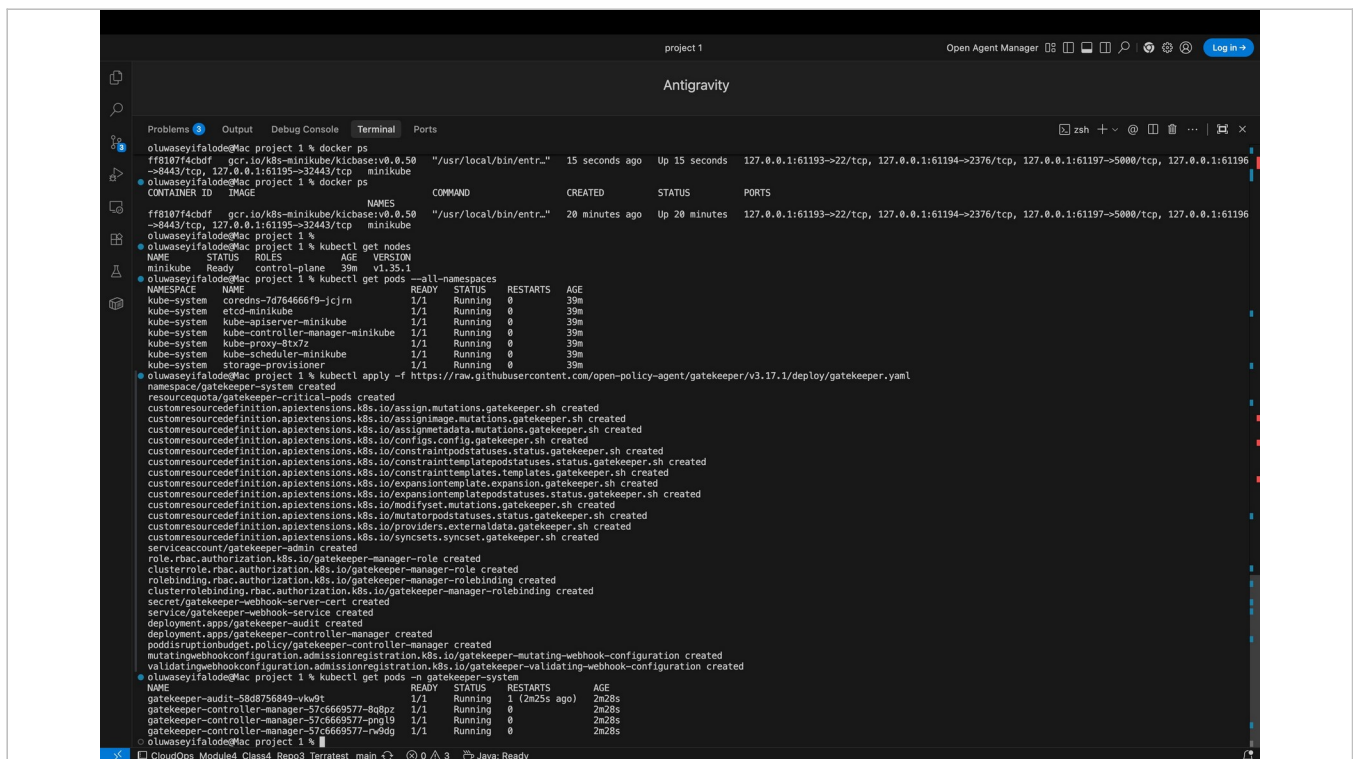
Key concepts explained

- **Admission webhook:** A Kubernetes mechanism that allows an external system to intercept API requests before they are processed. Gatekeeper registers itself as a validating admission webhook — meaning Kubernetes must ask Gatekeeper for approval before creating any resource.
- **Policy-as-Code:** Writing security rules as code files that are version-controlled and automatically enforced. Instead of a human manually reviewing every deployment, the rules run automatically every time. No human delay, no inconsistency.
- **Rego:** The policy language used by OPA. You describe what a violation looks like and OPA checks every incoming request against your description.
- **ConstraintTemplate:** The file that defines the rule logic in Rego. This is where you write what a violation means.
- **Constraint:** The file that activates a ConstraintTemplate in specific namespaces. This is where you say where and when to enforce the rule.

Step 3A — Installing Gatekeeper

Gatekeeper v3.17.1 was installed using a single `kubectl apply` command pointing to the official release manifest. This created over 20 resources including custom resource definitions, RBAC roles, service accounts, deployments, and — most critically — the validating webhook configuration that plugs Gatekeeper into Kubernetes' request pipeline.

```
kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper/v3.17.1/deploy/gatekeeper.yaml
```



```
project 1
Open Agent Manager
Antigravity

Problems Output Debug Console Terminal Ports
oluwaseyifalode@Mac project 1 % docker ps
ff8107f4cbdf gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr..." 15 seconds ago Up 15 seconds 127.0.0.1:61193->22/tcp, 127.0.0.1:61194->2376/tcp, 127.0.0.1:61197->5000/tcp, 127.0.0.1:61196->8443/tcp, 127.0.0.1:61195->32443/tcp minikube
oluwaseyifalode@Mac project 1 % docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
ff8107f4cbdf gcr.io/k8s-minikube/kicbase:v0.0.50 "/usr/local/bin/entr..." 20 minutes ago Up 20 minutes 127.0.0.1:61193->22/tcp, 127.0.0.1:61194->2376/tcp, 127.0.0.1:61197->5000/tcp, 127.0.0.1:61196->8443/tcp, 127.0.0.1:61195->32443/tcp minikube
oluwaseyifalode@Mac project 1 % kubectl get nodes
NAME STATUS ROLES AGE VERSION
minikube Ready control-plane 39m v1.35.1
oluwaseyifalode@Mac project 1 % kubectl get pods -all-namespaces
NAMESPACE NAME READY STATUS RESTARTS AGE
kube-system coredns-7d76466f9-jcjr 1/1 Running 0 39m
kube-system etcd-minikube 1/1 Running 0 39m
kube-system kube-apiserver-minikube 1/1 Running 0 39m
kube-system kube-controller-manager-minikube 1/1 Running 0 39m
kube-system kube-proxy-8t7z 1/1 Running 0 39m
kube-system kube-scheduler-minikube 1/1 Running 0 39m
kube-system storage-provisioner 1/1 Running 0 39m
oluwaseyifalode@Mac project 1 % kubectl apply -f https://raw.githubusercontent.com/open-policy-agent/gatekeeper/v3.17.1/deploy/gatekeeper.yaml
namespace/gatekeeper-system created
resourcequota/gatekeeper-critical-pods created
customresourcedefinition.apiextensions.k8s.io/assign.mutations.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/assignimage.mutations.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/assignmetadata.mutations.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/configs.config.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/constraintpodstatuses.status.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/constrainttemplatepodstatuses.status.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/expansiontemplates.templates.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/expansiontemplatepodstatuses.status.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/expansiontemplatepodstatuses.status.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/mutatorpodstatuses.status.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/providers.externaldata.gatekeeper.sh created
customresourcedefinition.apiextensions.k8s.io/syncsets.syncset.gatekeeper.sh created
serviceaccount/gatekeeper-admin created
role.rbac.authorization.k8s.io/gatekeeper-manager-role created
clusterrole.rbac.authorization.k8s.io/gatekeeper-manager-role created
rolebinding.rbac.authorization.k8s.io/gatekeeper-manager-rolebinding created
clusterrolebinding.rbac.authorization.k8s.io/gatekeeper-manager-rolebinding created
secret/gatekeeper-webhook-server-cert created
service/gatekeeper-webhook-service created
deployment.apps/gatekeeper-audit created
poddisruptionbudget.policy/gatekeeper-controller-manager created
mutatingwebhookconfiguration.admissionregistration.k8s.io/gatekeeper-mutating-webhook-configuration created
validatingwebhookconfiguration.admissionregistration.k8s.io/gatekeeper-validating-webhook-configuration created
oluwaseyifalode@Mac project 1 % kubectl get pods -n gatekeeper-system
NAME READY STATUS RESTARTS AGE
gatekeeper-audit-58d8756849-vk9t 1/1 Running 1 (2m25s ago) 2m28s
gatekeeper-controller-manager-57c6669577-b9lpz 1/1 Running 0 2m28s
gatekeeper-controller-manager-57c6669577-png19 1/1 Running 0 2m28s
gatekeeper-controller-manager-57c6669577-rv9dg 1/1 Running 0 2m28s
oluwaseyifalode@Mac project 1 %
```

Figure 6: Gatekeeper v3.17.1 installed: 20+ resources created including validatingwebhookconfiguration. All 4 gatekeeper-system pods Running. Admission webhook active.

Step 3B — Setting up the project folder

A dedicated project folder was created to organise all policy files. The folder was opened in the IDE (Antigravity — a VS Code-based editor) for writing the YAML policy files.

```
mkdir ~/devsecops-project && cd ~/devsecops-project
mkdir gatekeeper # subfolder for all policy files
```

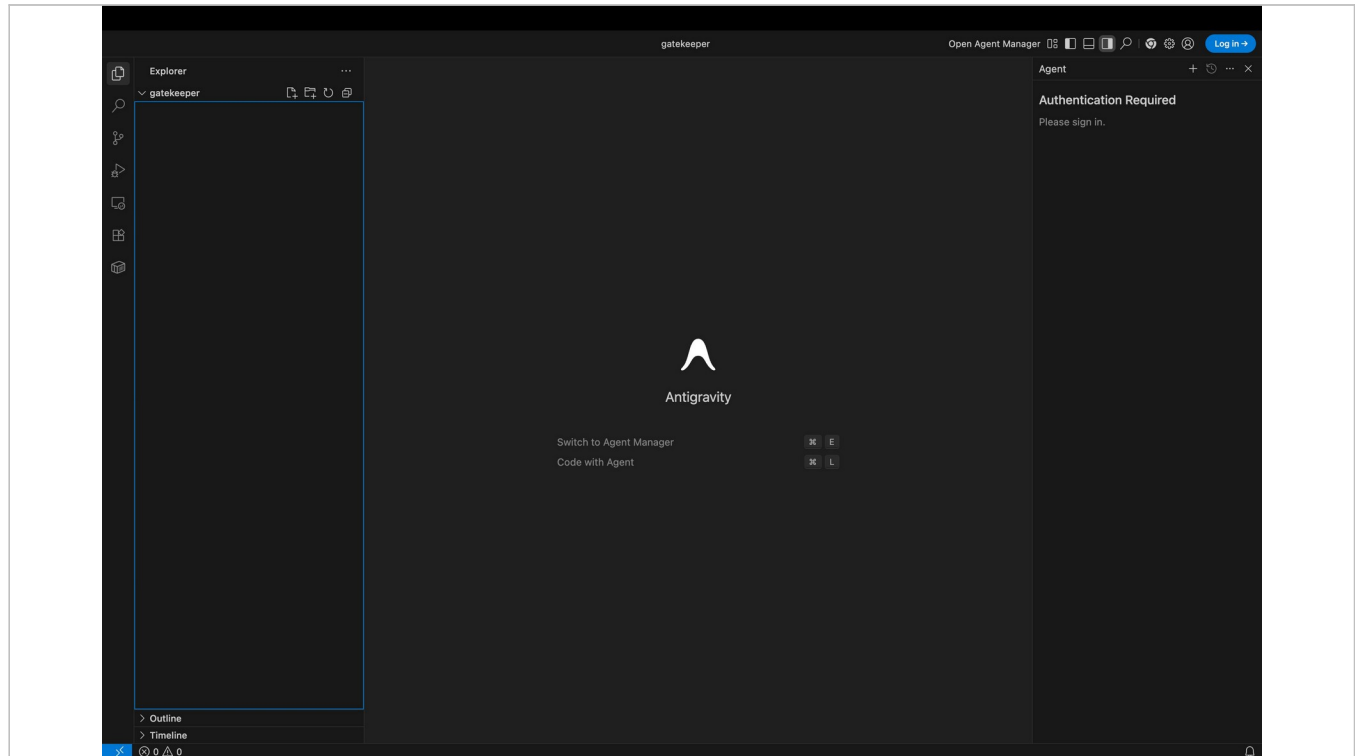


Figure 7: Project folder structure created: ~/devsecops-project/gatekeeper/ opened in Antigravity IDE. Ready to write policy files.

STEP 4 — Writing the No-Root Security Policy

What we are doing in this step

The first security policy is being written from scratch. The policy enforces that no container in the cluster is allowed to run as the root user (user ID 0). This is one of the most fundamental container security controls.

Why running as root is dangerous

In Linux, user ID 0 is the root user — the administrator with unrestricted access to everything on the system. When a container process runs as root, it has that same unrestricted access inside the container. If an attacker exploits a vulnerability in the application and gains code execution, they immediately have root-level privileges.

More critically, container isolation is not perfect. Vulnerabilities called container escapes allow a process inside a container to break out and access the host system. If that process is running as root inside the container, it lands on the host machine as root — with full control over the underlying server.

The principle of least privilege states that every process should run with only the minimum permissions it needs to do its job. Running a web server as root is never necessary — it can always run as a regular user with limited permissions.

Step 4A — Writing the ConstraintTemplate

The ConstraintTemplate defines the rule logic. It is written in Rego, OPA's policy language. The logic is: look at every container in the incoming deployment request. If any container has `runAsUser` set to 0 (root), that is a violation. Return a human-readable error message.

```
# File: gatekeeper/constraint-template.yaml
apiVersion: templates.gatekeeper.sh/v1
kind: ConstraintTemplate
metadata:
  name: k8snorootcontainers
spec:
  crd:
    spec:
      names:
        kind: K8sNoRootContainers
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8snorootcontainers
        violation[{"msg": msg}] {
          c := input.review.object.spec.containers[_]
          c.securityContext.runAsUser == 0
          msg := sprintf("BLOCKED: Container '%v' runs as root (UID 0)", [c.name])
```

Step 4B — Writing the Constraint

The Constraint activates the ConstraintTemplate. It specifies where the rule applies — in this case, to all Pods in the cluster, excluding the `kube-system` and `gatekeeper-system` namespaces. Those namespaces are excluded because they contain Kubernetes' own internal pods, some of which need elevated permissions to run.

```
# File: gatekeeper/constraint.yaml
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sNoRootContainers
metadata:
  name: deny-root-containers
spec:
  match:
    kinds:
      - apiGroups: [""]
        kinds: ["Pod"]
  excludedNamespaces:
```

```
- kube-system      # Kubernetes internal pods
- gatekeeper-system # Gatekeeper's own pods
```

Step 4C — Applying the policies to the cluster

Both files were applied to the cluster using `kubectl apply`. From this point forward, every pod deployment in the default namespace is checked against this policy automatically.

```
kubectl apply -f ~/devsecops-project/gatekeeper/constraint-template.yaml
kubectl apply -f ~/devsecops-project/gatekeeper/constraint.yaml
```

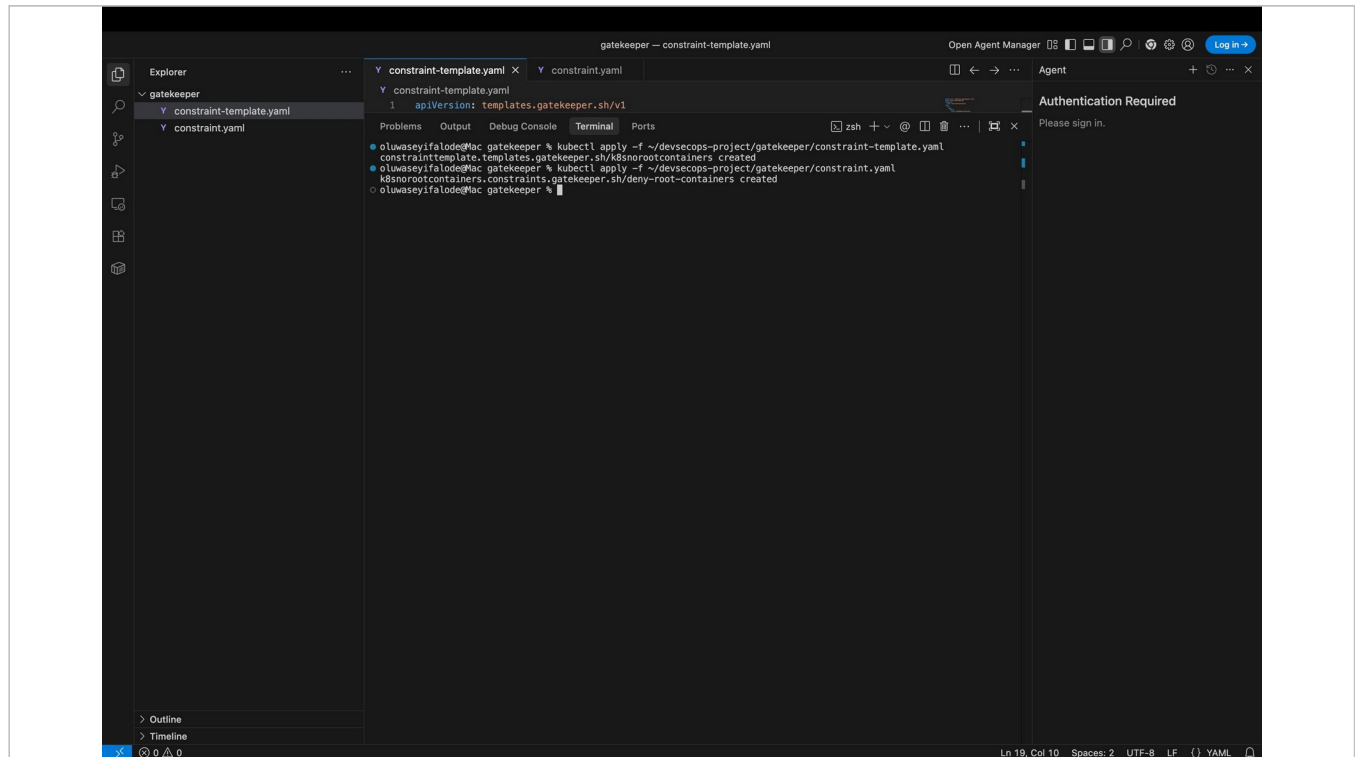


Figure 8: Both policy files applied successfully. constrainttemplate k8snorootcontainers created ✓ constraint deny-root-containers created ✓ Policy now active.

STEP 5 — Testing Policy Enforcement

What we are doing in this step

The policy must be tested to prove it works correctly. Two pod configurations were created — one deliberately violating the policy (to confirm it gets blocked) and one following the policy correctly (to confirm it is allowed through). A security control that blocks everything is useless. A control that blocks nothing is also useless. The test proves precision.

Test A — Deploying a bad pod (runAsUser: 0)

The first test pod was configured with runAsUser: 0 — explicitly setting the container to run as the root user. This is a direct violation of the policy. The pod configuration file looked like this:

```
# File: gatekeeper/bad-pod.yaml
apiVersion: v1
kind: Pod
metadata:
  name: bad-pod
  namespace: default
spec:
  containers:
  - name: bad-container
    image: nginx
    securityContext:
      runAsUser: 0 # ← ROOT USER — should be blocked
```

When `kubectl apply` was run with this file, Gatekeeper intercepted the request before Kubernetes could act on it. The error message returned was the exact message written in the policy:

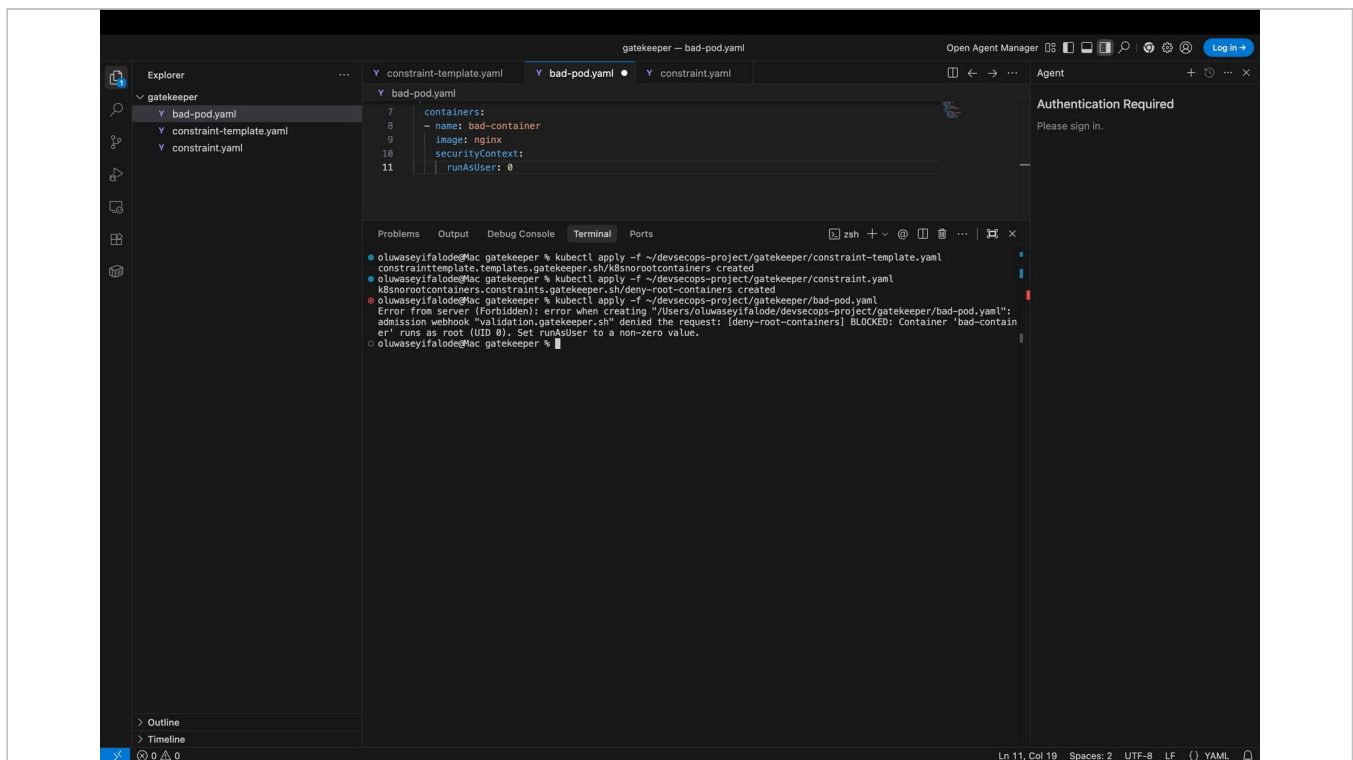


Figure 9: Bad pod BLOCKED: admission webhook "validation.gatekeeper.sh" denied the request. Error: BLOCKED: Container bad-container runs as root (UID 0). Pod never created.

The pod was never created. It never touched the cluster scheduler. Kubernetes rejected it at the entrance before any resource was allocated.

Test B — Deploying a good pod (runAsUser: 1000)

The second test pod used runAsUser: 1000 — a standard non-root user. This should pass Gatekeeper's check. However, the standard nginx image was used first, which caused an unexpected error that demonstrates an important real-world lesson.

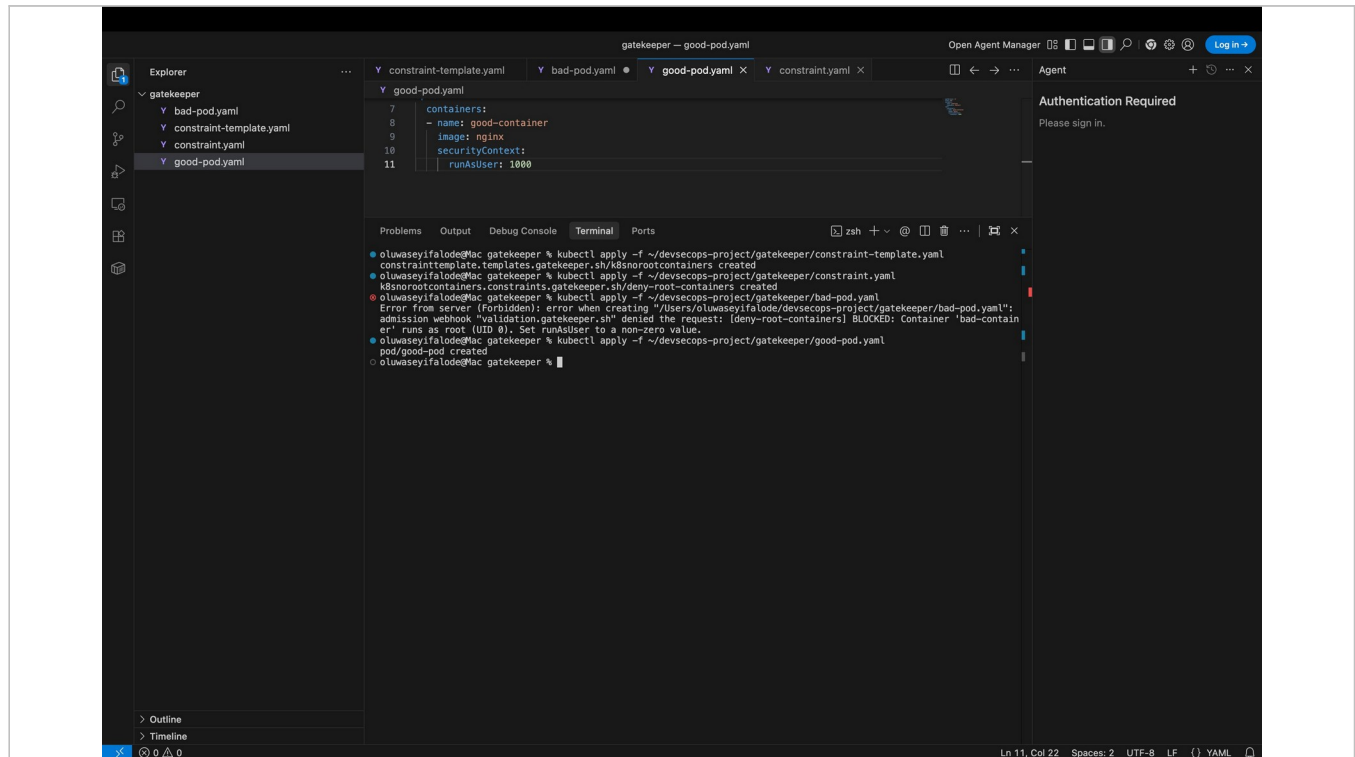


Figure 10: Good pod passed Gatekeeper (pod/good-pod created) but entered CrashLoopBackOff — nginx requires root to bind to port 80, which user 1000 cannot do.

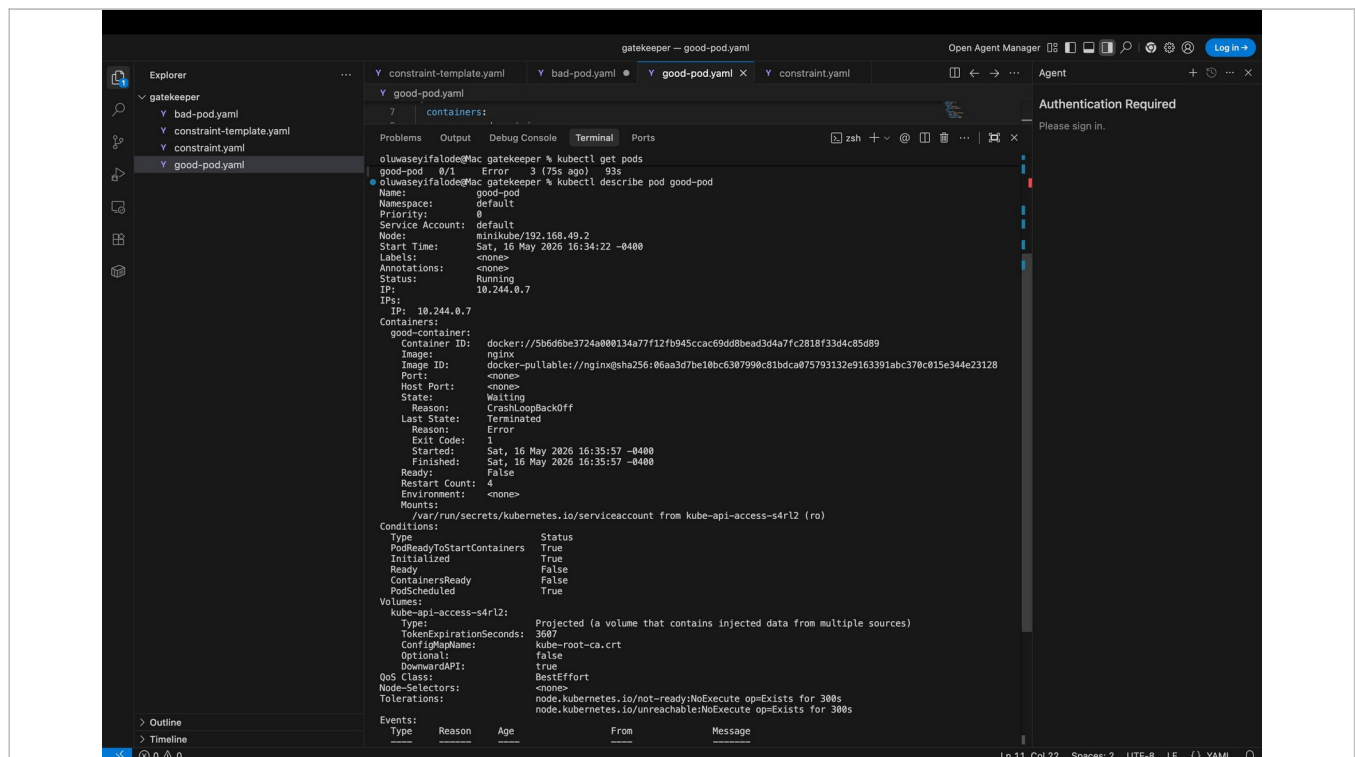


Figure 11: kubectl describe pod good-pod: CrashLoopBackOff confirmed. Exit Code: 1. Root cause — standard nginx cannot bind port 80 without root privileges.

This is an important real-world security lesson: even a correctly configured container can fail if the wrong image is chosen. The standard nginx image was built assuming it will run as root. The fix is to use `nginxinc/nginx-unprivileged` — a version of nginx specifically built to run as a non-root user, binding to port 8080 instead of 80.

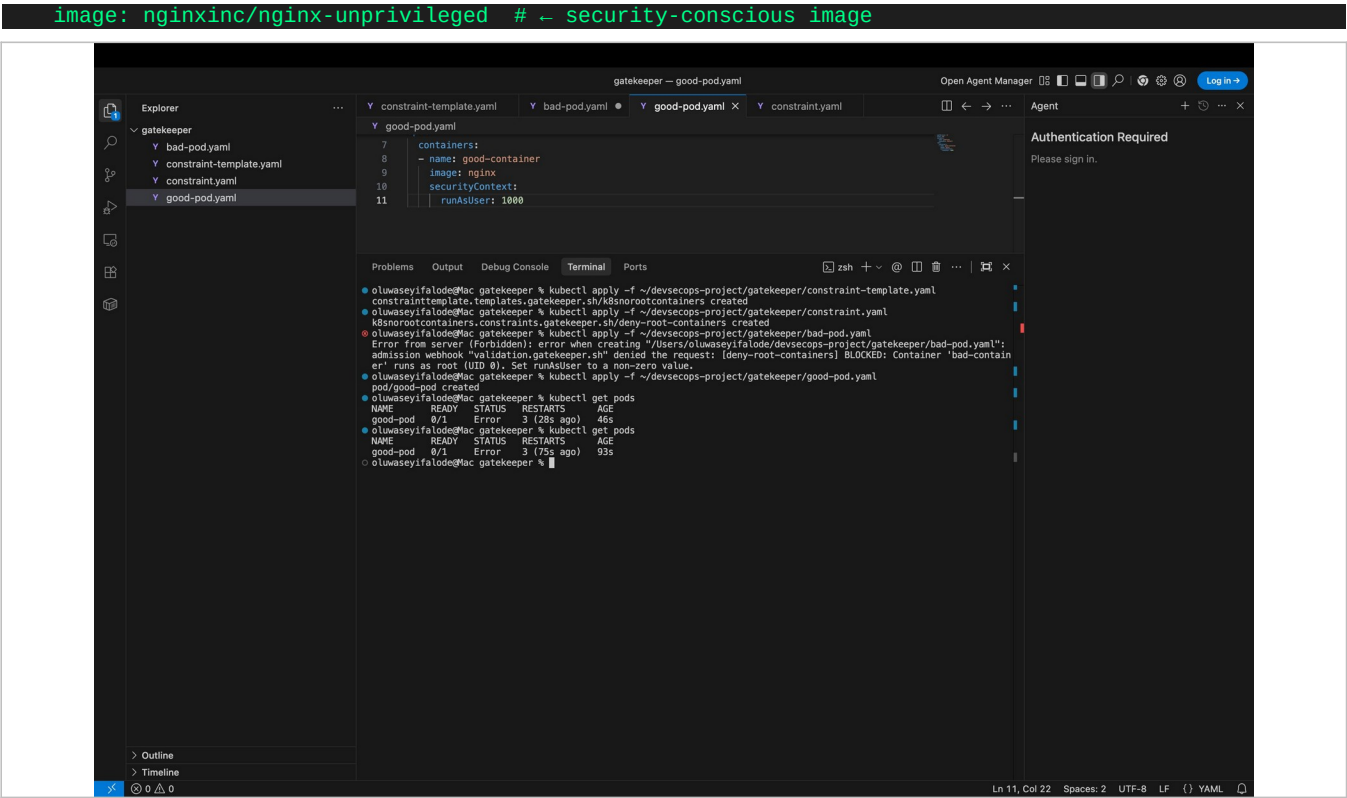


Figure 12: Final good pod: `nginxinc/nginx-unprivileged` with `runAsUser: 1000`. STATUS=Running, READY=1/1, RESTARTS=0. Policy passed, container healthy.

Test results summary

Pod	Configuration	Result
bad-pod (nginx)		runAsUser: 0 (root)
good-pod v1 (nginx)		runAsUser: 1000 — wrong image
good-pod v2 (nginx-unprivileged)		runAsUser: 1000 — correct image

STEP 6 — Trivy Image Vulnerability Scanning

What we are doing in this step

Trivy is used to scan container images for known security vulnerabilities before they are deployed. This represents Layer 1 of the pipeline — catching problems at the image level, before the image ever reaches the cluster.

Key concepts explained

- **CVE (Common Vulnerabilities and Exposures):** A publicly maintained database of known software vulnerabilities. Each entry has a unique ID (e.g. CVE-2026-27456), a description of the vulnerability, a severity score, and information about affected versions and fixes. Security researchers, software vendors, and attackers all use this same database.
- **Severity levels:** CRITICAL (actively exploited, fix immediately), HIGH (significant risk, fix soon), MEDIUM (moderate risk), LOW (minimal risk). In a production pipeline, CRITICAL and HIGH findings automatically fail the build.
- **Why images have vulnerabilities:** Container images are built on top of a base operating system — commonly Debian, Ubuntu, or Alpine Linux. The OS comes with hundreds of pre-installed software packages. Each of those packages may have known vulnerabilities. The image owner may not update them frequently, leaving old vulnerable versions in place.

Step 6A — Scanning the standard nginx image

The standard nginx image from Docker Hub was scanned. This is one of the most widely used container images in the world, pulled billions of times. The results were significant.

trivy image nginx

```
gatekeeper — good-pod.yaml
1 apiVersion: v1
2 kind: Pod
3 metadata:
4   name: good-pod
5   namespace: default
6 spec:
```

Problems Output Debug Console Terminal Ports

zsh + + @ ...

```
oluwaseyifalade@gatekeeper % trivy image nginx 2>/dev/null | head -50
Report Summary
┌──────────┬──────────┬──────────┬──────────┐
│ Target    │ Type     │ Vulnerabilities │ Secrets │
├──────────┼──────────┼──────────┬──────────┤
│ nginx (debian 13.4) │ debian │ 233 │ - │
└──────────┴──────────┴──────────┴──────────┘

Legend:
- '!': Not scanned
- '0': Clean (no security findings detected)

nginx (debian 13.4)
Total: 233 (UNKNOWN: 7, LOW: 115, MEDIUM: 86, HIGH: 22, CRITICAL: 3)

┌──────────┬──────────┬──────────┬──────────┬──────────┬──────────┐
│ Library   │ Vulnerability Title │ Severity │ Status │ Installed Version │ FI │
├──────────┼──────────┬──────────┼──────────┼──────────┬──────────┤
│ apt       │ CVE-2011-3374      │ LOW      │ affected │ 3.0.3 │ │
│           │ It was found that apt-key in apt, all versions, do not correctly... │ │ │ │ │ │
│           │ https://avd.aquasec.com/nvd/cve-2011-3374 │ │ │ │ │ │
├──────────┼──────────┬──────────┼──────────┼──────────┬──────────┤
│ bash      │ TEMP-0841856-B18BAF │ MEDIUM   │ affected │ 5.2.37-2+b8 │ │
│           │ [Privilege escalation possible to other user than root] │ │ │ │ │ │
│           │ https://security-tracker.debian.org/tracker/TEMP-0841856-B1-8BAF │ │ │ │ │ │
├──────────┼──────────┬──────────┼──────────┼──────────┬──────────┤
│ bsdutils  │ CVE-2026-27456     │ MEDIUM   │ affected │ 1:2.41-5 │ │
│           │ util-linux: TOCTOU in the mount program when setting up loop │ │ │ │ │ │
│           │ devices │ │ │ │ │ │
│           │ https://avd.aquasec.com/nvd/cve-2026-27456 │ │ │ │ │ │
└──────────┴──────────┴──────────┴──────────┴──────────┴──────────┘
```

Ln 11, Col 22 Spaces: 2 UTF-8 LF {} YAML

Figure 13: Trivy scan — nginx (debian 13.4): Total: 233 vulnerabilities. CRITICAL: 3 HIGH: 22 MEDIUM: 86 LOW: 115 UNKNOWN: 7

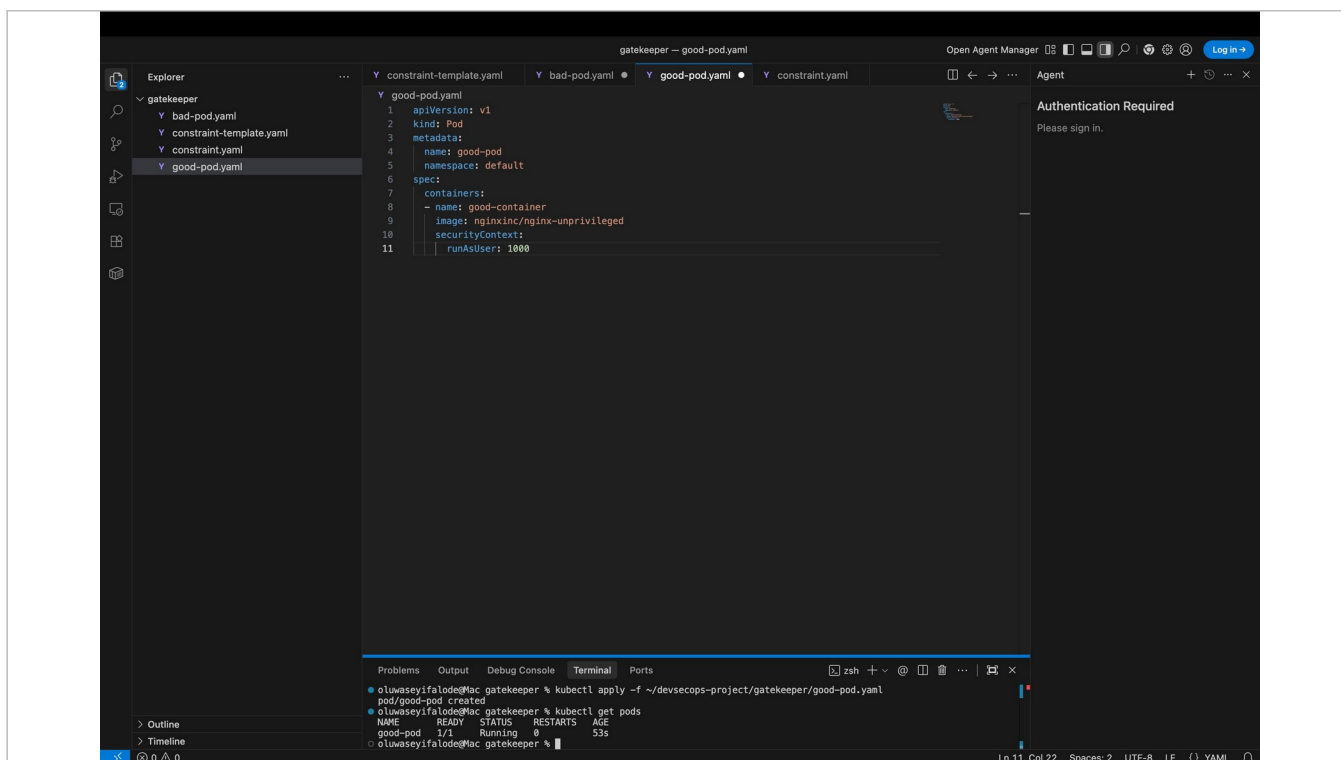


Figure 14: Trivy CVE detail view: individual vulnerability entries showing CVE ID, severity, affected package name, description, and fix version.

Notable finding: the bash package inside the nginx image has a privilege escalation vulnerability. An attacker who gains any level of access inside the container could potentially escalate their privileges using this vulnerability.

Step 6B — Scanning the nginx-unprivileged image for comparison

The nginxinc/nginx-unprivileged image was scanned next to compare its security posture against the standard nginx image.

```
trivy image nginxinc/nginx-unprivileged 2>/dev/null | grep 'Total:'
```

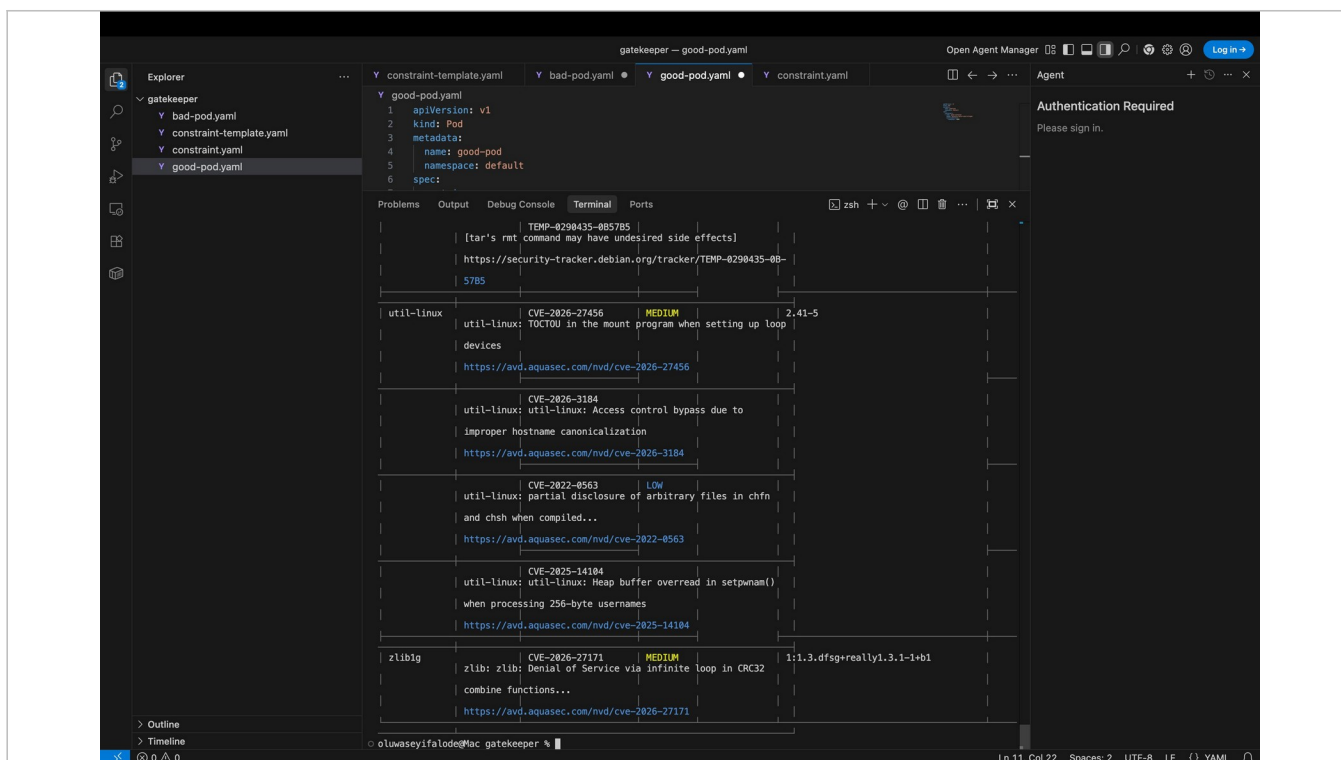


Figure 15: Trivy scan — `nginxinc/nginx-unprivileged`: Total: 232 vulnerabilities. CRITICAL: 3 HIGH: 21 MEDIUM: 86 Nearly identical to standard `nginx`.

Scan comparison and analysis

Image	CRITICAL	HIGH	MEDIUM	TOTAL
nginx (standard)	3	22	86	233
nginxinc/nginx-unprivileged	3	21	86	232

Both images have nearly identical vulnerability counts because they are both built on Debian Linux. The vulnerabilities come from the underlying operating system packages, not from `nginx` itself. A truly hardened image would use Alpine Linux as the base OS — Alpine is a minimal Linux distribution that includes far fewer packages, dramatically reducing the attack surface.

STEP 7 — Falco Runtime Threat Detection

What we are doing in this step

Falco is being installed and tested as the third and final layer of the security pipeline. While Trivy and Gatekeeper prevent problems before and at deployment, Falco watches what actually happens inside containers while they are running. This is runtime security.

Key concepts explained

- **System call (syscall):** Every action a program takes at the operating system level is done through a system call — reading a file, writing to disk, opening a network connection, creating a new process. These are the lowest-level building blocks of all computer activity.
- **eBPF (extended Berkeley Packet Filter):** A Linux kernel technology that allows programs to run safely inside the kernel without modifying its source code. Falco uses eBPF to attach small monitoring programs to every syscall event. This gives it complete visibility into container activity with minimal performance overhead.
- **Why runtime detection matters:** Gatekeeper checks the configuration. But what if the configuration is fine and an attacker exploits a vulnerability in the running application? The app might have a bug that allows code injection. The attacker opens a shell inside the container to explore the system. Falco detects this the moment it happens, regardless of how the attacker got in.
- **Helm:** The Kubernetes package manager. Used to install Falco because Falco has multiple components that need to be deployed and configured together. Helm handles all of that automatically.

Step 7A — Installing Falco

Three commands were run to install Falco. First, the Falco chart repository was added to Helm. Second, the repository was updated to fetch the latest package information. Third, Falco was installed into its own namespace with the modern eBPF driver enabled.

```
# Step 1: Add Falco chart repository to Helm
helm repo add falcosecurity https://falcosecurity.github.io/charts

# Step 2: Update repository
helm repo update

# Step 3: Install Falco with eBPF driver
helm install falco falcosecurity/falco \
  --namespace falco \           # install in its own namespace
  --create-namespace \         # create the namespace if it doesn't exist
  --set driver.kind=modern_ebpf \ # use eBPF for kernel monitoring
  --set tty=true                # readable terminal output
```

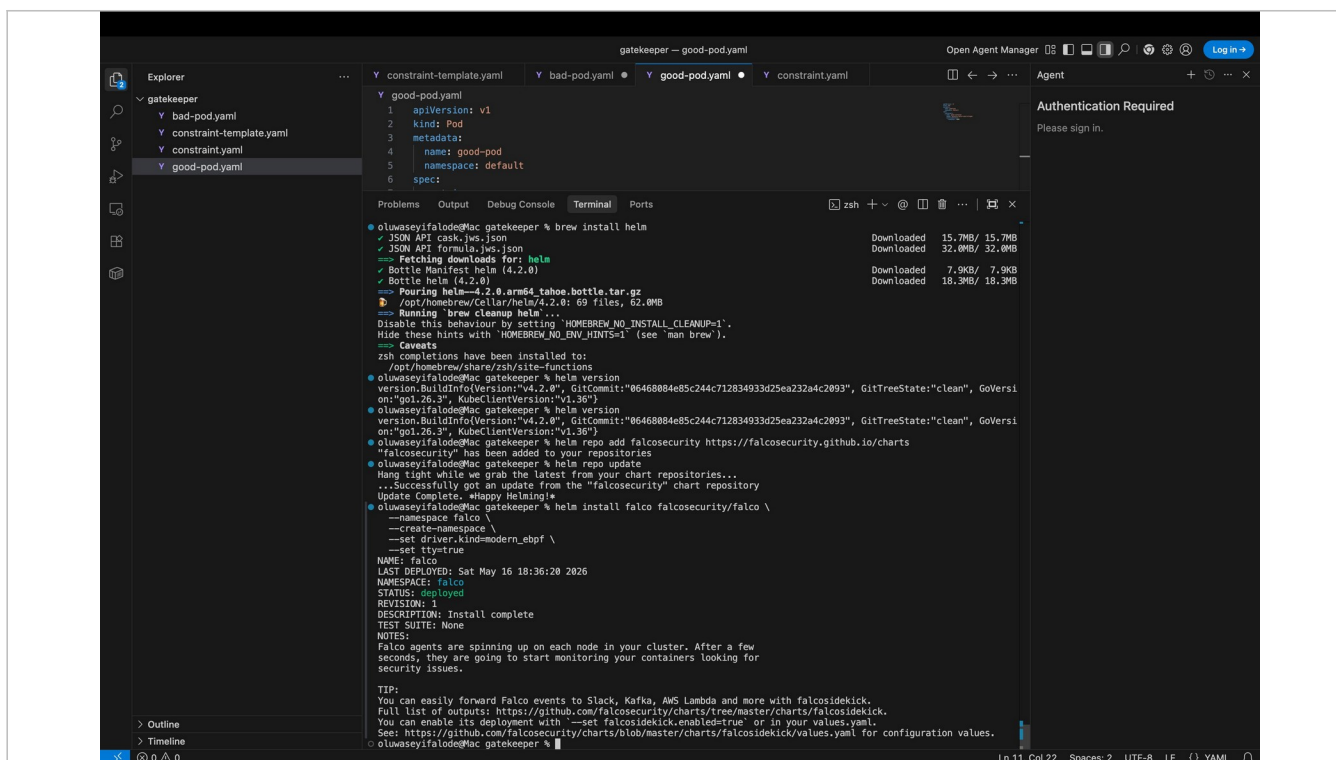


Figure 16: Falco deployed: STATUS=deployed, DESCRIPTION=Install complete. falco pod Running 2/2. Falco agents spinning up and monitoring all containers.

Step 7B — Triggering a live alert

To demonstrate that Falco is actively monitoring, a deliberate trigger was executed: an interactive shell was opened inside the running good-pod container using `kubectl exec`. In a real attack scenario, this is one of the most common post-exploitation actions — an attacker who has gained access to a container opens a shell to run commands and explore the environment.

Falco has a built-in rule called 'Terminal shell in container' that fires the moment any shell process spawns inside a container with an attached terminal. This is exactly what `kubectl exec` does.

```
# Open an interactive shell inside the running container
kubectl exec -it good-pod -- /bin/sh
$ whoami      # ran inside the container — shows user 1000
$ exit        # exited the container shell
```

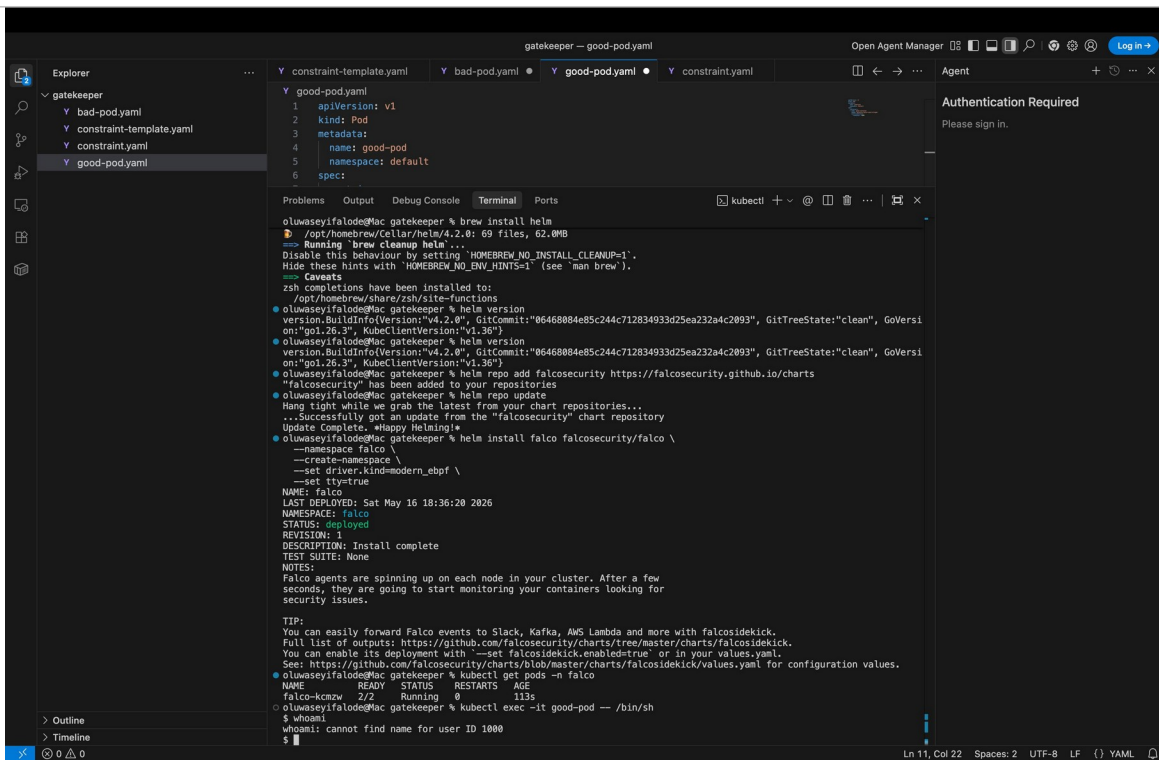


Figure 17: `kubectl exec` opened a shell inside `good-pod`. `whoami` returned user ID 1000 (non-root as configured). Falco detected this syscall event in real time.

Step 7C — Reading the Falco alert

Immediately after exiting the container, the Falco logs were checked. Falco had already captured and logged the shell spawn event with complete forensic detail.

```
kubectl logs -n falco -l app.kubernetes.io/name=falco --tail=20
```

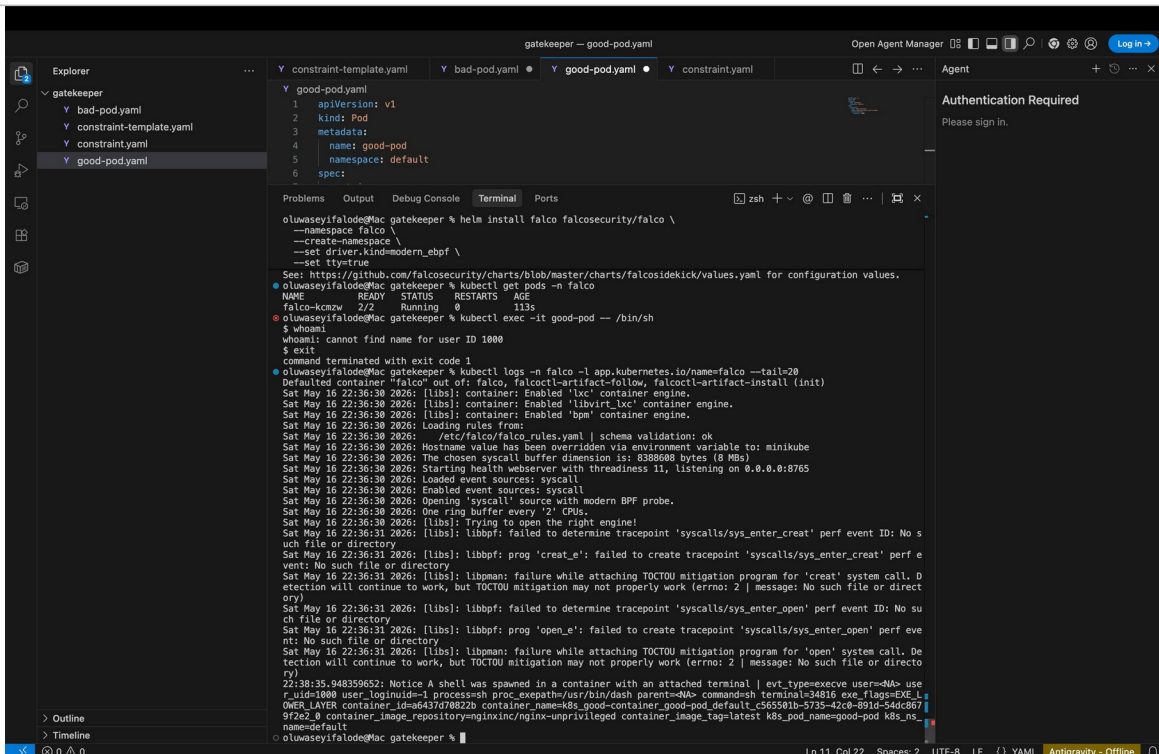


Figure 18: Falco alert captured: "Notice A shell was spawned in a container with an attached terminal." Metadata: user_uid=1000, process=sh, container=good-pod, image=nginx-unprivileged, namespace=default.

The alert contains everything a security operations team needs to investigate and respond: who triggered it (user ID 1000), what process was spawned (sh), which container (good-pod), which image (nginx-unprivileged), and which namespace (default). In a production environment this alert would be forwarded to a SIEM platform (Splunk, Elasticsearch) and could trigger an automated response — isolating the pod or cordoning the node — within seconds.

STEP ✓ — Project Complete — Full Results

What was built

A complete three-layer Kubernetes container security pipeline, built from scratch on a local Minikube cluster. All three layers were deployed, configured, and tested with real results documented throughout.

Deliverable	Outcome
Kubernetes cluster	Minikube v1.38.1 — single node, Kubernetes v1.35.1 — fully operational
OPA Gatekeeper	v3.17.1 deployed — validating webhook active — 20+ resources created
No-root policy	ConstraintTemplate + Constraint applied — enforced in default namespace
Bad pod test	runAsUser: 0 rejected at admission layer — pod never created or scheduled
Good pod test	nginx-unprivileged + runAsUser: 1000 — Running 1/1, 0 restarts
Trivy scan — nginx	233 vulnerabilities found: CRITICAL: 3, HIGH: 22, MEDIUM: 86
Trivy scan — unprivileged	232 vulnerabilities found: CRITICAL: 3, HIGH: 21 — same Debian base OS
Falco deployment	Helm install — STATUS: deployed — falco pod Running 2/2 — eBPF active
Falco live alert	Shell spawn detected in <3 seconds — full forensic metadata logged

How to describe this project in an interview

"I built a three-layer Kubernetes container security pipeline using OPA Gatekeeper, Trivy, and Falco on a local Minikube cluster. I wrote ConstraintTemplates in Rego to enforce a no-root policy — tested it by deliberately deploying a non-compliant pod configured as root, and Gatekeeper rejected it at the admission webhook before the pod was ever scheduled. I then ran Trivy against the nginx image and found 233 vulnerabilities including three critical severity CVEs embedded in the Debian base OS packages. Finally I deployed Falco using Helm with the modern eBPF driver, opened a shell inside a running container to simulate a post-exploitation scenario, and Falco fired a runtime alert in under three seconds with complete pod and namespace metadata. All three layers were tested and documented with real terminal evidence."